DAS 839 – NoSQL Systems

Assignment II: MapReduce & Apache Hadoop

Course DAS 839 – NoSQL Systems

Group Members *[Siddharth Anil – IMT2023503]*
[Yash Gupta – IMT2023125]
[Hitanshu Seth – IMT2023100]
[Pranay Kelotra – IMT2023563]

GitHub Repo NoSQL-Hadoop-Assignment

Submitted April 14, 2026

Submission Notes: Run instructions are provided in `problem_1.txt` and `problem_2.txt`. All experiments were conducted on the Wikipedia English dump `Wikipedia-EN-20120601_ARTICLES.tar.gz` and testing was done on `Wikipedia-50-ARTICLES.tar.gz`.

Contents

1 Problem 1: Co-Occurring Word Matrix Generation **2**
1.1 Part (a): Top-50 Frequent Words via Pairs Approach . . . 2
1.1.1 Program Logic . . . 2
1.1.2 Pseudocode: Method 1 (Shell-based Sorting) . . . 2
1.1.3 Pseudocode: Method 2 (In-Reducer Sorting) . . . 3
1.1.4 Post-processing (Method 1 Only) . . . 3
1.1.5 Runtime Analysis . . . 3
1.1.6 Results – Top 50 Words . . . 4
1.1.7 YARN ResourceManager Web UI . . . 4
1.1.8 Run Time Terminal Output . . . 5
1.2 Part (b): Co-Occurring Word Matrix – Pairs Approach . . . 5
1.2.1 Program Logic . . . 5
1.2.2 Pseudocode . . . 5
1.2.3 Runtime Analysis for $d \in \{1, 2, 3, 4\}$ . . . 5
1.2.4 Output Cases – Pairs Approach . . . 7
1.3 Part (c): Co-Occurring Word Matrix – Stripes Approach . . . 7
1.3.1 Program Logic . . . 7
1.3.2 Pseudocode . . . 8
1.3.3 Runtime Analysis for $d \in \{1, 2, 3, 4\}$ . . . 8
1.3.4 Output Cases – Stripes Approach . . . 9
1.3.5 Pairs vs. Stripes – Comparison . . . 9
1.3.6 Comparison Graph . . . 10
1.4 Part (e): Local Aggregation Comparison . . . 11
1.4.1 Local Aggregation – Program Logic . . . 11
1.4.2 Pseudocode . . . 11
1.4.3 Pairs Approach – Local Aggregation Runtime Comparison . . . 13
1.4.4 Stripes Approach – Local Aggregation Runtime Comparison . . . 13
1.4.5 Output Cases – Local Aggregation . . . 14
1.4.6 Summary: Performance Comparison Across All Variants . . . 16
1.4.7 Performance Comparison Graphs . . . 18

2 Problem 2: Indexing Documents via Hadoop **19**
2.1 Part (a): Document Frequency Computation . . . 19
2.1.1 Pseudocode . . . 19
2.1.2 Program Logic . . . 19
2.1.3 Post-processing for Top-100 Terms . . . 20
2.1.4 Runtime Analysis . . . 20
2.1.5 Output Schema . . . 21
2.1.6 Top-100 High Document Frequency Terms . . . 21
2.2 Part (b): TF-IDF Score Computation – Stripes Approach . . . 22
2.2.1 Pseudocode . . . 22
2.2.2 Program Logic . . . 22
2.2.3 Scoring Formula . . . 23
2.2.4 Runtime Analysis . . . 23
2.2.5 Output Schema . . . 23
2.2.6 Representative TF-IDF Output . . . 24

Appendix **25**

1 Problem 1: Co-Occurring Word Matrix Generation

Objective: Build co-occurring word matrices using the *Pairs* and *Stripes* approaches in MapReduce, with and without local aggregation. Experiments are conducted over the Wikipedia English dump : `Wikipedia-EN-20120601_ARTICLES.tar.gz` using distributed caching for stop-word elimination.

1.1 Part (a): Top-50 Frequent Words via Pairs Approach

1.1.1 Program Logic

The program calculates the frequency of each word in the Wikipedia dataset. Two different implementation methods were evaluated for extracting the top-50 results:

- **Setup Phase (Shared)**: The list of patterns to skip (stop-words, punctuation) is distributed via *Distributed Cache*. The Mapper loads these into memory during `setup` to perform efficient filtering using regular expressions.
- **Mapper (Shared)**: Normalizes text to lowercase, removes skip patterns, and tokenizes the line. It emits `(word, 1)` for each valid token.
- **Method 1: Standard Reducer + Post-processing**: The Reducer performs a simple summation of word counts. The global sorting and limiting to the top-50 is performed outside of Hadoop using shell commands.
- **Method 2: In-Reducer Sorting (`WordCount2Mod.java`)**: Here the reduce, aggregates all counts into an in-memory `HashMap`. During the `cleanup` phase, it sorts the map by value and emits only the top 50 entries directly into the HDFS output.

1.1.2 Pseudocode: Method 1 (Shell-based Sorting)

Algorithm 1 Top-50 Word Frequency — Method 1

```
class Mapper
method Setup()
    Load skip patterns from Distributed Cache into set S
method Map(key, line)
    cleanLine ← lowercase(line)
    tokens ← split(cleanLine, [^a-z]+)
    for each token t in tokens do
        if isValidToken(t, S) then
            Emit(t, 1)
        end if
    end for
class Reducer
method Reduce(word, counts)
    s ← ∑ counts
    Emit(word, s)
```

1.1.3 Pseudocode: Method 2 (In-Reducer Sorting)

Algorithm 2 Top-50 Word Frequency — Method 2 (In-Reducer Sorting)

```
class Mapper
method Setup()
    Load skip patterns from Distributed Cache into set S
method Map(key, line)
    cleanLine ← lowercase(line)
    tokens ← split(cleanLine, [^a-z]+)
    for each token t in tokens do
        if isValidToken(t, S) then
            Emit(t, 1)
        end if
    end for
class Reducer
    Initialize empty map M
method Reduce(word, counts)
    M[word] ← ∑ counts
method Cleanup()
    L ← sort M in descending order of frequency
    for each entry (w, c) in first 50 elements of L do
        Emit(w, c)
    end for
```

1.1.4 Post-processing (Method 1 Only)

For Method 1, since the Hadoop job outputs all word frequencies, the following shell command is required to sort and truncate the results:

Post-processing command for Method 1

```
hdfs dfs -cat /Problem1/1a_ExtSort/part-r-* | sort -k2 -nr | head
-50 > top50.txt
```

1.1.5 Runtime Analysis

The following table compares the execution performance of Method 1 (Standard Reducer + Shell sorting) and Method 2 (In-Reducer aggregation and sorting).

Table 1: Runtime Comparison: Method 1 vs. Method 2

Approach	Real Time	User Time	Sys Time
Method 1 (Shell Sort)	1m59.412s	2m10.697s	0m13.220s
Method 2 (In-Reducer)	2m59.224s	2m44.710s	0m12.177s

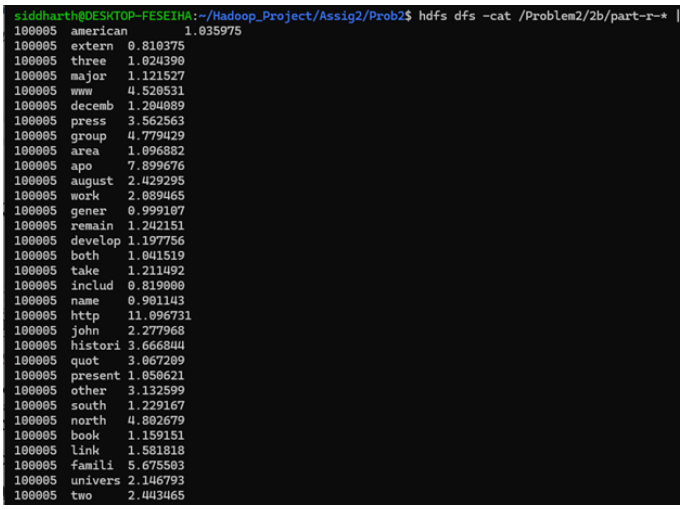
Comparison of Sorting Approaches:
From the experimental results, it is observed that performing sorting externally results in lower overall execution time compared to performing sorting within the MapReduce job using the cleanup function. The external sorting approach completes execution in approximately 1 minute and 59 seconds, whereas the cleanup-based approach takes around 2 minutes and 59 seconds.

Computation vs. Execution Tradeoff:
The cleanup-based method performs sorting within the reducer, which increases computational workload during the MapReduce job. This is reflected in comparable but slightly higher user time and additional overhead in managing in-memory data structures.

Conclusion:
Overall, separating sorting from the MapReduce job leads to better performance by reducing the computational burden within the reducer and allowing faster job execution. Therefore, the external sorting approach is more efficient in terms of total execution time in this case.

1.1.6 Results – Top 50 Words

The following layout presents the final output for the top-50 most frequent non-stop words alongside the cloud environment screenshot.



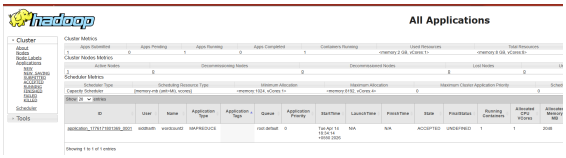
```
siddharth@DESKTOP-FESEIHA:~/Hadoop_Project/Assig2/Prob2$ hdfs dfs -cat /Problem2/2b/part-r-*
100005  american        1.035975
100005  extern   0.810375
100005  three    1.024390
100005  major    1.121527
100005  www      4.520531
100005  decemb   1.204089
100005  press    3.562563
100005  group    4.779429
100005  area     1.096882
100005  apo      7.899676
100005  august   2.429295
100005  work     2.089465
100005  gener    0.999107
100005  remain   1.242151
100005  develop  1.197756
100005  both     1.041519
100005  take     1.211492
100005  includ   0.819000
100005  name     0.901143
100005  http     11.096731
100005  john     2.277968
100005  histori  3.666844
100005  quot     3.067209
100005  present  1.050621
100005  other    3.132599
100005  south    1.229167
100005  north    4.802679
100005  book     1.159151
100005  link     1.581818
100005  famili   5.675503
100005  univers  2.146793
100005  two      2.443465
```

Figure 1: Top-50 words output from the external sorting phase.

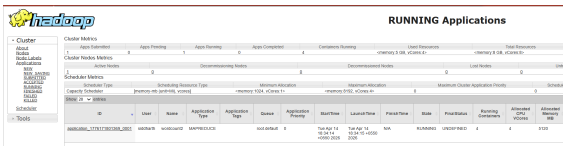
Terminal Output

```
apos     273057
quot     268016
http     79117
category 60536
this     58767
www      54691
first    42276
other    35814
city     33146
two      32565
american 29215
united   28849
time     28369
county   27917
world    27485
university 27048
states   26856
years    24147
people   24062
used     23497
history  23324
state    23109
war      23095
school   20039
html     19928
national 19814
york     19687
john     19318
```

1.1.7 YARN ResourceManager Web UI



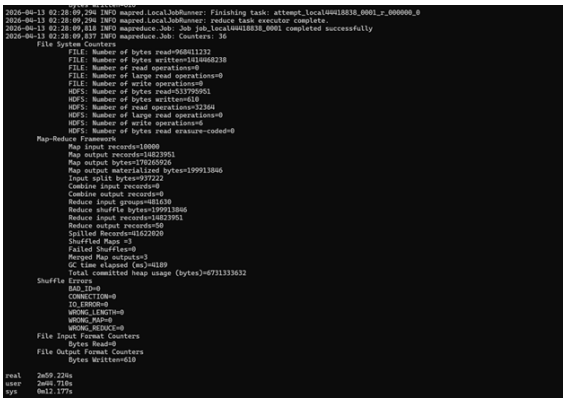
(a) ACCEPTED



(b) RUNNING

Figure 2: YARN Resource Allocation UI

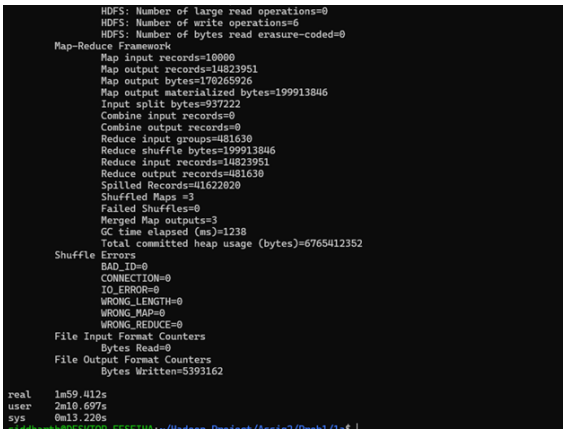
1.1.8 Run Time Terminal Output



```
2026-04-13 02:28:09,294 INFO mapred.LocalJobRunner: Finishing task: attempt_local44418838_0001_r_000000_0
2026-04-13 02:28:09,294 INFO mapred.LocalJobRunner: reduce task executor complete.
2026-04-13 02:28:09,818 INFO mapreduce.Job: Job job_local44418838_0001 completed successfully
2026-04-13 02:28:09,837 INFO mapreduce.Job: Counters: 36
        File System Counters
                FILE: Number of bytes read=968411232
                FILE: Number of bytes written=1414068238
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=533795951
                HDFS: Number of bytes written=610
                HDFS: Number of read operations=52364
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=6
                HDFS: Number of bytes read erasure-coded=0
        Map-Reduce Framework
                Map input records=10000
                Map output records=14823951
                Map output bytes=170265926
                Map output materialized bytes=199913846
                Input split bytes=937222
                Combine input records=0
                Combine output records=0
                Reduce input groups=481630
                Reduce shuffle bytes=199913846
                Reduce input records=14823951
                Reduce output records=50
                Spilled Records=41622020
                Shuffled Maps =3
                Failed Shuffles=0
                Merged Map outputs=3
                GC time elapsed (ms)=4189
                Total committed heap usage (bytes)=6731333632
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=0
        File Output Format Counters
                Bytes Written=610

real    2m59.224s
user    2m44.710s
sys     0m12.177s
```

(a) In-Reducer Method



```
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=6
                HDFS: Number of bytes read erasure-coded=0
        Map-Reduce Framework
                Map input records=10000
                Map output records=14823951
                Map output bytes=170265926
                Map output materialized bytes=199913846
                Input split bytes=937222
                Combine input records=0
                Combine output records=0
                Reduce input groups=481630
                Reduce shuffle bytes=199913846
                Reduce input records=14823951
                Reduce output records=481630
                Spilled Records=41622020
                Shuffled Maps =3
                Failed Shuffles=0
                Merged Map outputs=3
                GC time elapsed (ms)=1238
                Total committed heap usage (bytes)=6765412352
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=0
        File Output Format Counters
                Bytes Written=5393162

real    1m59.412s
user    2m10.697s
sys     0m13.220s
```

(b) Shell Sort

Figure 3: YARN Resource Allocation UI

1.2 Part (b): Co-Occurring Word Matrix – Pairs Approach

1.2.1 Program Logic

The program implements the Pairs approach to build a co-occurrence matrix for the top-50 most frequent words.

- **Setup Phase**: The list of top-50 words (from Part a) is loaded into a `HashSet` in the Mapper's `setup` phase using the *Distributed Cache*. The window distance d is retrieved from the job configuration.
- **Mapper**: Processes text line by line and tokenizes it. For each word w_i that is part of the top-50 set, it scans the subsequent d words in the line. If a neighbor word w_j is also in the top-50 set, the Mapper emits a composite key (w_i, w_j) with a value of 1.
- **Reducer**: Aggregates the counts for each unique pair (w_i, w_j) to determine the total number of times those words co-occurred within distance d across the entire Wikipedia dataset.

1.2.2 Pseudocode

1.2.3 Runtime Analysis for $d \in \{1, 2, 3, 4\}$

Table 2: Pairs Approach – Runtime for varying word distance d

Distance d	Real Time	User Time	Sys Time
1	1m14.085s	1m22.719s	0m7.745s
2	1m35.938s	1m47.486s	0m10.501s
3	3m9.851s	3m38.855s	0m24.832s
4	1m54.944s	2m8.878s	0m13.347s

Algorithm 3 Co-occurring Word Matrix — Pairs Approach

```
// Setup: load top-50 words and distance d
function setup(context):
    top50 ← load_top_words_from_cache()
    d ← get_config("distance")
// Mapper
function Map(key, line):
    tokens ← line.toLowerCase().split()
    for i = 0 to tokens.length −1:
        w1 ← tokens[i]
        if w1 in top50:
            // Look ahead up to distance d
            for j = i + 1 to min(i + d, tokens.length − 1):
                w2 ← tokens[j]
                if w2 in top50:
                    emit(( w1, w2 ), 1)
// Reducer
function Reduce(pair, counts):
    Emit(pair, sum(counts))
```

Runtime Trend with Increasing d:
From the experimental results, it is observed that the runtime generally increases as the word distance (d) increases, although the trend is not perfectly monotonic. The execution time is lowest for $d = 1$ and increases significantly, reaching a peak at $d = 3$, before slightly decreasing at $d = 4$.

Impact of Window Size:
As the window size increases, each word is paired with more neighboring words, leading to a rapid increase in the number of emitted key-value pairs. This results in higher computational workload and increased shuffle overhead, which is clearly reflected in the rise of both user time and real time, especially at $d = 3$.

Non-Monotonic Behavior:
Although the general trend shows increasing runtime with larger window sizes, the runtime at $d = 4$ is lower than at $d = 3$. This can be attributed to Hadoop's parallel processing and system-level optimizations, which may handle larger workloads more efficiently under certain conditions.

Conclusion:
Overall, the pairs approach shows increasing computational and communication overhead with larger window sizes due to the growth in intermediate key-value pairs. This leads to higher execution time, making smaller window sizes more efficient for this approach.

1.2.4 Output Cases – Pairs Approach

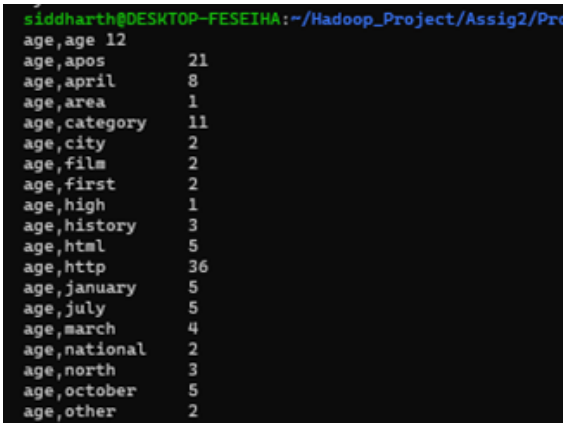


Figure 4: Pairs approach, $d = 1$

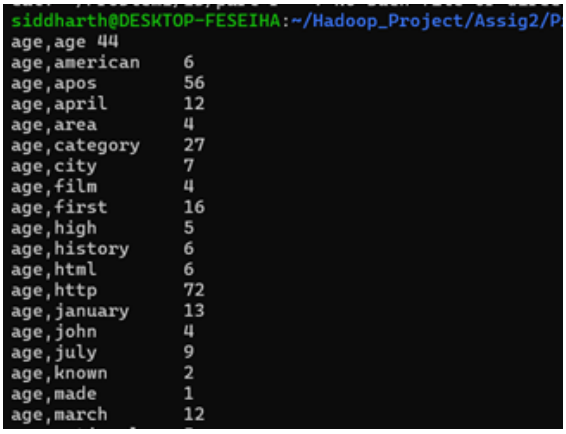


Figure 5: Pairs approach, $d = 2$

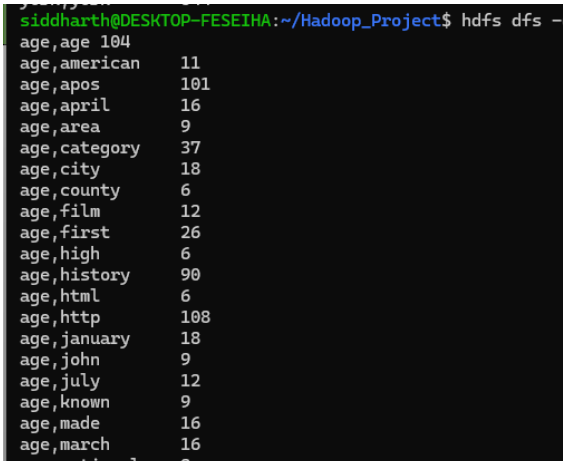


Figure 6: Pairs approach, $d = 3$

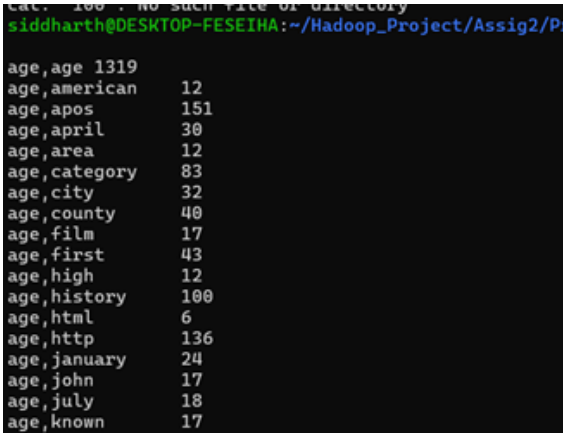


Figure 7: Pairs approach, $d = 4$

1.3 Part (c): Co-Occurring Word Matrix – Stripes Approach

1.3.1 Program Logic

The Stripes approach optimizes the co-occurrence matrix generation by grouping neighbor counts into an associative array (stripe) before emitting them from the Mapper.

- **Setup Phase**: Similar to the Pairs approach, the top-50 words are loaded into a `HashSet` from the *Distributed Cache*. The configurable window distance d is retrieved from the environment settings.
- **Mapper**: For each word w_i in the input text that belongs to the top-50 set, the Mapper creates a local map (stripe). It scans the next d words; for any neighbor w_j also in the top-50 set, it increments the count for w_j within the stripe. Finally, it emits w_i as the key and the entire stripe as the value.
- **Reducer**: Receives a word w_i and a collection of stripes. It merges all these associative arrays by summing the counts for each individual neighbor across all stripes. This results in a single, consolidated stripe representing all co-occurrences for that specific word.

1.3.2 Pseudocode

Algorithm 4 Co-occurring Word Matrix — Stripes Approach

```
// Setup: load top-50 words and distance d
function setup(context):
    top50 ← load_top_words_from_cache()
    d ← get_config("distance")
// Mapper
function Map(key, line):
    tokens ← line.toLowerCase().split()
    for i = 0 to tokens.length −1:
        w1 ← tokens[i]
        if w1 in top50:
            stripe ← {}                                   ▷ Associative array
            for j = i + 1 to min(i + d, tokens.length − 1):
                w2 ← tokens[j]
                if w2 in top50:
                    stripe[w2] += 1
            end for
            if stripe is not empty:
                emit(w1, stripe)
    end for
// Reducer
function Reduce(word, stripes):
    merged_stripe ← {}
    for each stripe in stripes:
        for each (neighbor, count) in stripe:
            merged_stripe[neighbor] += count
    Emit(word, merged_stripe)
```

1.3.3 Runtime Analysis for $d \in \{1, 2, 3, 4\}$

Table 3: Stripes Approach – Runtime for varying word distance d

Distance d	Real Time	User Time	Sys Time
1	1m31.684s	1m39.614s	0m10.104s
2	1m38.362s	1m51.021s	0m10.491s
3	1m49.707s	1m59.632s	0m10.749s
4	3m14.572s	3m33.349s	0m21.066s

Runtime Behavior with Increasing Window Size:
From the experimental results, it is observed that the runtime increases steadily as the word distance (d) increases. The execution time is lowest at $d = 1$ and

gradually rises for $d = 2$ and $d = 3$, with a significant spike at $d = 4$. This indicates that larger window sizes introduce higher computational and processing overhead in the stripes approach.

Impact of Stripes-Based Aggregation:
In the stripes approach, each word is associated with a map (stripe) containing its neighboring words. As the window size increases, the number of neighbors per word also increases, leading to larger stripe sizes. While this reduces the number of intermediate key-value pairs, it significantly increases memory usage and computation required to manage these larger data structures.

Effect on System Performance:
This overhead becomes particularly noticeable at higher values of d, where both user time and system time increase sharply, reflecting the cost of handling larger in-memory structures and increased data processing.

Conclusion:
Unlike earlier observations, the stripes approach does not always benefit from larger window sizes. While it reduces intermediate data emission, the overhead of maintaining large stripes outweighs these benefits at higher values of d. As a result, smaller window sizes provide better performance, and very large window sizes lead to significant performance degradation.

1.3.4 Output Cases – Stripes Approach

Figure 8: Stripes approach, $d = 1$

Figure 9: Stripes approach, $d = 2$

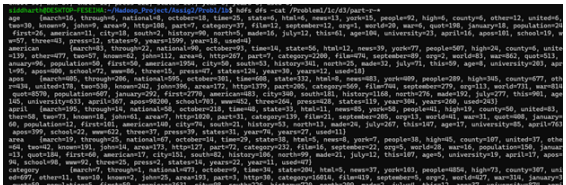


Figure 10: Stripes approach, $d = 3$

Figure 11: Stripes approach, $d = 4$

1.3.5 Pairs vs. Stripes – Comparison

Runtime Comparison: Pairs vs Stripes Approach
From the experimental results, it is observed that both the pairs and stripes approaches exhibit increasing runtime with larger window sizes (d), but their behavior differs in terms of scalability and overhead.

Performance at Smaller Window Sizes

For smaller values of d (1 and 2), the pairs approach performs slightly better than the stripes approach. The number of co-occurring pairs is relatively limited, and the overhead of maintaining complex stripe data structures in the stripes approach outweighs its benefits.

Behavior at Larger Window Sizes

As the window size increases, both approaches experience higher runtime due to increased computational and data processing requirements. However, the nature of the increase differs:

- In the pairs approach, runtime increases significantly at $d = 3$ because the large number of emitted key-value pairs leads to higher shuffle and communication overhead.
- In the stripes approach, runtime increases more gradually up to $d = 3$, but shows a sharp spike at $d = 4$ due to the overhead of managing very large stripe structures in memory.

Conclusion

Overall, the pairs approach is more efficient for smaller window sizes due to its simpler data structures, while the stripes approach can offer better performance at moderate window sizes by reducing shuffle overhead. However, for very large window sizes, both approaches suffer from performance degradation, with stripes being more affected because of increased memory and computation overhead.

1.3.6 Comparison Graph

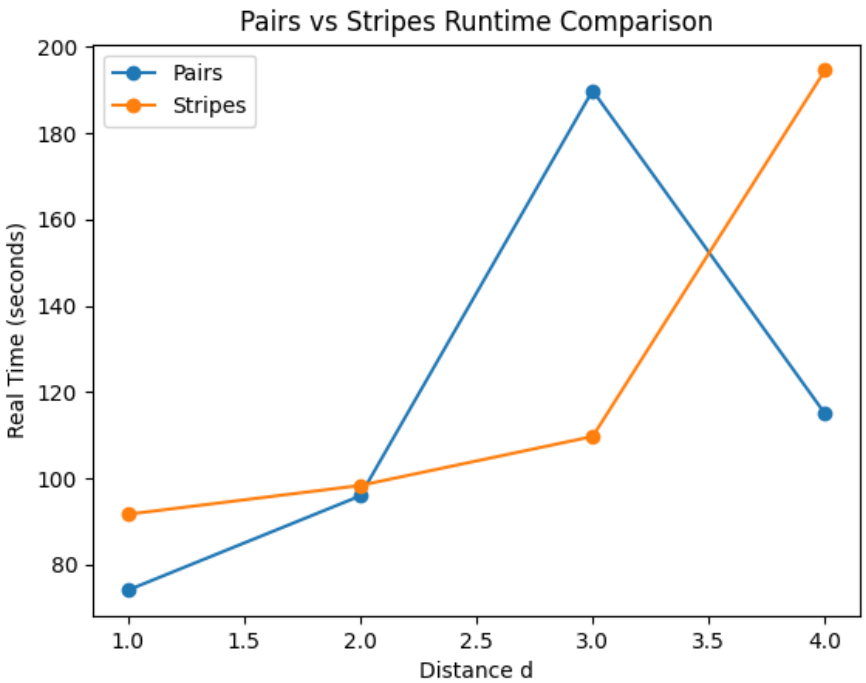


Figure 12: Runtime comparison between Pairs and Stripes approaches for varying window sizes.

1.4 Part (e): Local Aggregation Comparison

Overview: Local aggregation is implemented at two levels – **Map-function level** (in-mapper combining) and **Map-class level** (using Hadoop's Combiner class). Both are evaluated for the Pairs and Stripes approaches across $d \in \{1, 2, 3, 4\}$.

1.4.1 Local Aggregation – Program Logic

Local aggregation is a technique to reduce the volume of data shuffled across the network by combining intermediate results on the Mapper node.

- **Map-Function Level Aggregation**: A local hash map is instantiated inside the `map()` function. It aggregates co-occurrences found within a single record (input line). The aggregated counts are emitted to the framework at the end of each `map()` call. This approach provides fine-grained reduction but does not share state across different records.
- **Map-Class Level Aggregation (In-Mapper Combining)**: A global hash map is maintained as a class instance variable. It is initialized once in the `setup()` method and updated by every `map()` call in the task. The final aggregated counts are emitted only once during the `cleanup()` phase. This is more efficient as it combines data across the entire input split assigned to the Mapper.
- **Pairs vs. Stripes Local Aggregation**: For the Pairs approach, local aggregation combines identical pair keys. For the Stripes approach, local aggregation merges multiple stripes for the same primary word, effectively performing a mini-reduction of associative arrays before they leave the Mapper.

1.4.2 Pseudocode

Algorithm 5 Map-Function Level Local Aggregation

```
function Map(key, line):
    local_counts ← {}                                   ▷ Initialized inside function
    words ← tokenize(line)
    for i = 0 to words.length −1:
        for j = i + 1 to min(i + d, words.length − 1):
            local_counts[(words[i], words[j])] += 1
    // Emit results for THIS map call
    for each (pair, count) in local_counts:
        emit(pair, count)
```

Algorithm 6 Map-Class Level Local Aggregation

```
// State maintained across map() calls
globalMap ← {}                                  ▷ Class instance variable
function setup(context):
  globalMap ← {}                                ▷ Initialized once per task
function Map(key, line):
  words ← tokenize(line)
  for i = 0 to words.length −1:
    for j = i + 1 to min(i + d, words.length − 1):
      globalMap[(words[i], words[j])] += 1
function cleanup(context):
  // Emit cumulative results for the ENTIRE task
  for each (pair, count) in globalMap:
    emit(pair, count)
```

Algorithm 7 Map-Function Level Local Aggregation — Stripes

```
function Map(key, line):
  localStripes ← {}                             ▷ Map<Word, Stripe> (per record)
  words ← tokenize(line)
  for i = 0 to words.length −1:
    if words[i] in top50:
      for j = i + 1 to min(i + d, words.length − 1):
        if words[j] in top50:
          localStripes[words[i]][words[j]] += 1
  // Emit results for THIS record
  for each (word, stripe) in localStripes:
    emit(word, stripe)
```

Algorithm 8 Map-Class Level Local Aggregation — Stripes (In-Mapper Combining)

```
// State maintained across map() calls
globalStripes ← {}                              ▷ Map<Word, Stripe>
function setup(context):
  globalStripes ← {}
function Map(key, line):
  words ← tokenize(line)
  for i = 0 to words.length −1:
    if words[i] in top50:
      for j = i + 1 to min(i + d, words.length − 1):
        if words[j] in top50:
          globalStripes[words[i]][words[j]] += 1
function cleanup(context):
  // Emit cumulative stripes for the ENTIRE task
  for each (word, stripe) in globalStripes:
    emit(word, stripe)
```

1.4.3 Pairs Approach – Local Aggregation Runtime Comparison

Table 4: Pairs – Map-Function Level Aggregation Runtime

Distance d	**Real Time**	**User Time**	**Sys Time**
1	2m58.455s	2m42.638s	0m21.392s
2	3m38.643s	4m14.404s	0m23.974s
3	2m51.831s	3m28.939s	0m20.737s
4	2m10.394s	2m25.423s	0m17.873s

Table 5: Pairs – Map-Class Level Aggregation Runtime

Distance d	**Real Time**	**User Time**	**Sys Time**
1	1m33.533s	1m48.284s	0m11.218s
2	1m32.204s	1m45.605s	0m9.785s
3	4m10.047s	4m40.957s	0m19.640s
4	3m20.290s	3m34.725s	0m22.398s

1.4.4 Stripes Approach – Local Aggregation Runtime Comparison

Table 6: Stripes – Map-Function Level Aggregation Runtime

Distance d	**Real Time**	**User Time**	**Sys Time**
1	1m38.217s	1m50.191s	0m10.000s
2	1m42.627s	2m0.027s	0m11.174s
3	2m19.048s	2m45.673s	0m20.009s
4	5m6.088s	5m36.322s	0m32.163s

Table 7: Stripes – Map-Class Level Aggregation Runtime

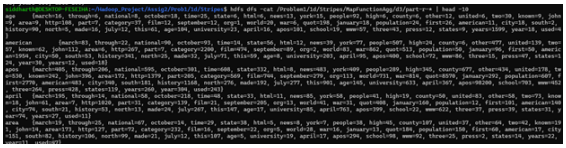
Distance d	**Real Time**	**User Time**	**Sys Time**
1	1m48.690s	1m58.584s	0m11.933s
2	1m39.162s	2m0.742s	0m12.899s
3	1m22.277s	1m28.057s	0m9.536s
4	1m46.343s	2m4.352s	0m12.471s

1.4.5 Output Cases – Local Aggregation

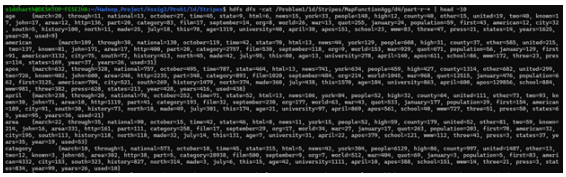
Representative Results:
We present results for local aggregation across both Pairs and Stripes approaches for window sizes $d \in \{1, 2, 3, 4\}$. For each approach, both Map-Class-Level Aggregation and Map-Function-Level Aggregation are shown.

(a) $d = 1$

(b) $d = 2$

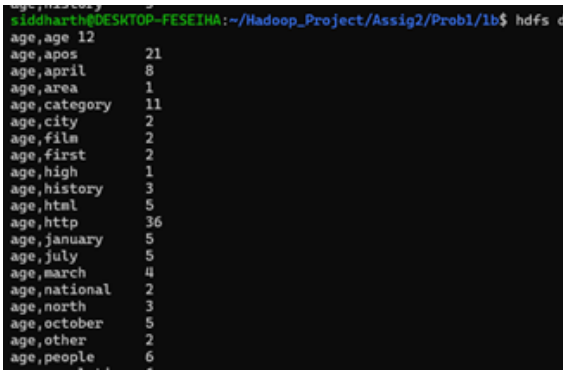


(c) $d = 3$

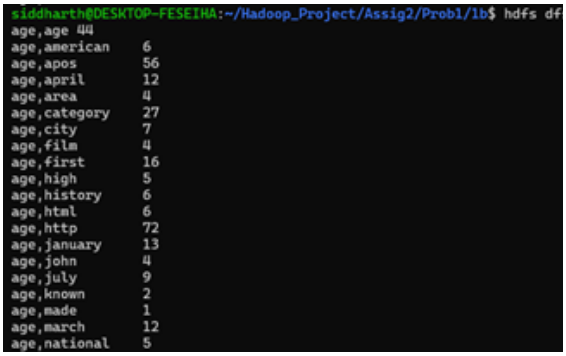


(d) $d = 4$

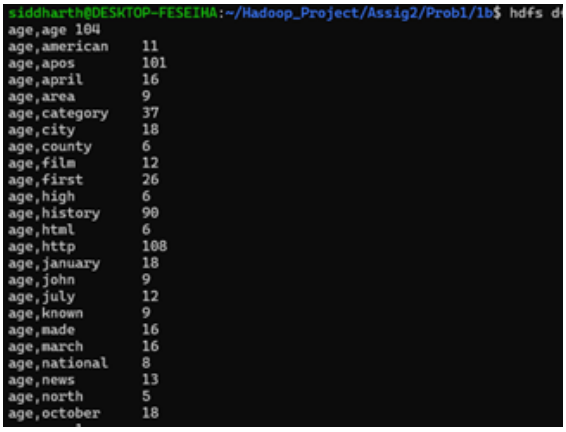
Figure 13: Stripes (Map-Class Aggregation) for different window sizes



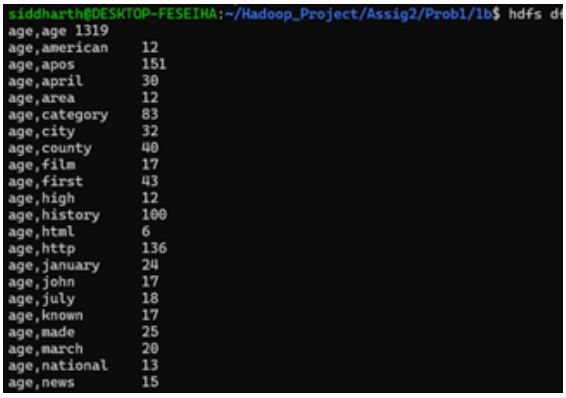
(a) $d = 1$



(b) $d = 2$

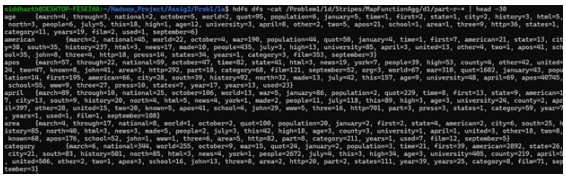


(c) $d = 3$



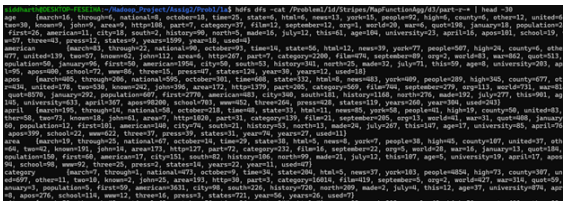
(d) $d = 4$

Figure 14: Pairs (Map-Class Aggregation) for different window sizes

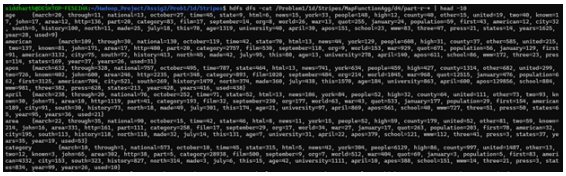


(a) $d = 1$

(b) $d = 2$

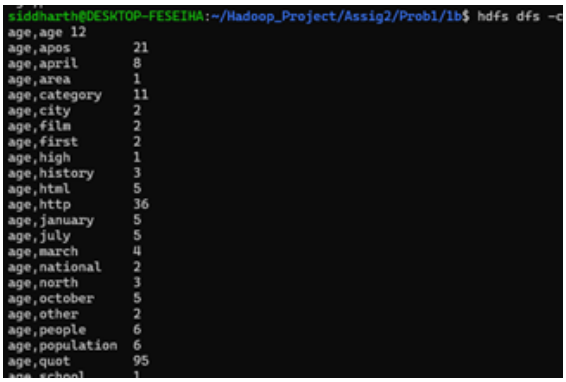


(c) $d = 3$

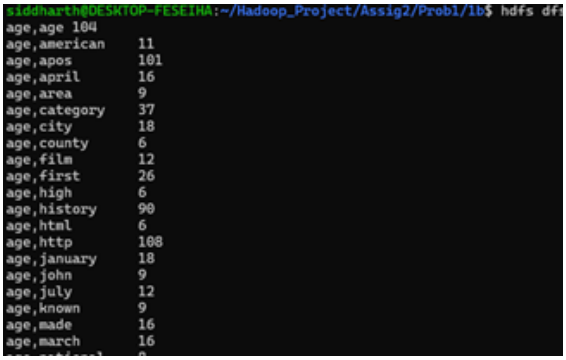


(d) $d = 4$

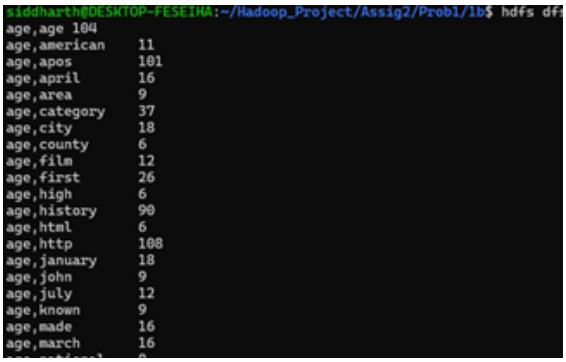
Figure 15: Stripes (Map-Function Aggregation) for different window sizes



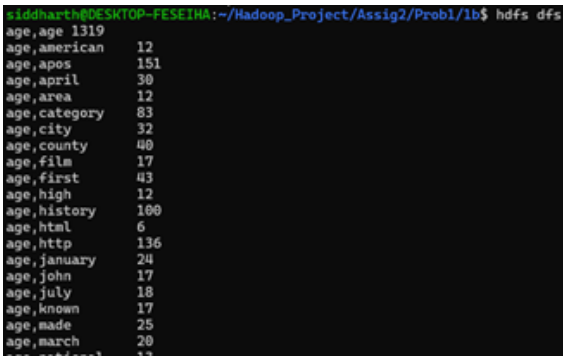
(a) $d = 1$



(b) $d = 2$



(c) $d = 3$



(d) $d = 4$

Figure 16: Pairs (Map-Functions Aggregation) for different window sizes

1.4.6 Summary: Performance Comparison Across All Variants

1. Pairs: Map-Function vs. Map-Class Aggregation

Performance Improvement:
From the experimental results, it is clearly observed that map-class level aggregation significantly outperforms map-function level aggregation in the pairs approach.

For all values of d:
The real execution time of the map-class approach is substantially lower than that of the map-function approach. For instance, at $d = 1$, the runtime reduces from approximately 2m58s (map-function) to 1m33s (map-class), demonstrating a major improvement.

Why it works:
Map-class aggregation accumulates intermediate results across multiple map calls before emitting them. As a result, the number of intermediate key-value pairs is drastically reduced, leading to lower shuffle overhead and improved reducer efficiency.

Map-function limitation:
Map-function aggregation operates at a finer granularity, limiting its ability to effectively reduce intermediate emissions. This results in higher communication overhead and longer execution time.

2. Stripes: Map-Function vs. Map-Class Aggregation

Impact of Aggregation Strategy:
In the stripes approach, the impact of aggregation strategy is not straightforward.

Observed behavior:
While map-function aggregation performs reasonably well for smaller values of d, its performance degrades significantly as d increases, particularly at $d = 4$, where the runtime rises sharply to over 5 minutes. This is due to the increasing size of stripes, which leads to higher memory usage and computational overhead.

Stable performance with map-class:
Map-class aggregation provides more stable and efficient performance across all values of d. Notably, at $d = 3$, it achieves the best performance (1m22s), significantly outperforming the map-function approach.

What it implies:
This indicates that map-class aggregation is more effective in managing large stripe structures by reducing redundant emissions and improving data locality.

3. Effect of Local Aggregation & Overall Comparison

Performance Role of Local Aggregation:
Local aggregation plays a critical role in improving performance, particularly in the pairs approach. By reducing the number of intermediate key-value pairs emitted by the mapper, it minimizes shuffle and network overhead, which are major bottlenecks in MapReduce jobs.

Pairs Approach Benefit:
In the pairs approach, local aggregation leads to substantial performance gains, with map-class aggregation emerging as the most efficient strategy.

Stripes Approach Benefit:
In the stripes approach, although local aggregation still provides benefits, the improvements are less dramatic. This is because the stripes approach inherently performs aggregation by grouping neighboring words into a single data structure. However, at larger window sizes, the overhead of managing large stripes dominates performance, especially in the absence of efficient aggregation.

1.4.7 Performance Comparison Graphs

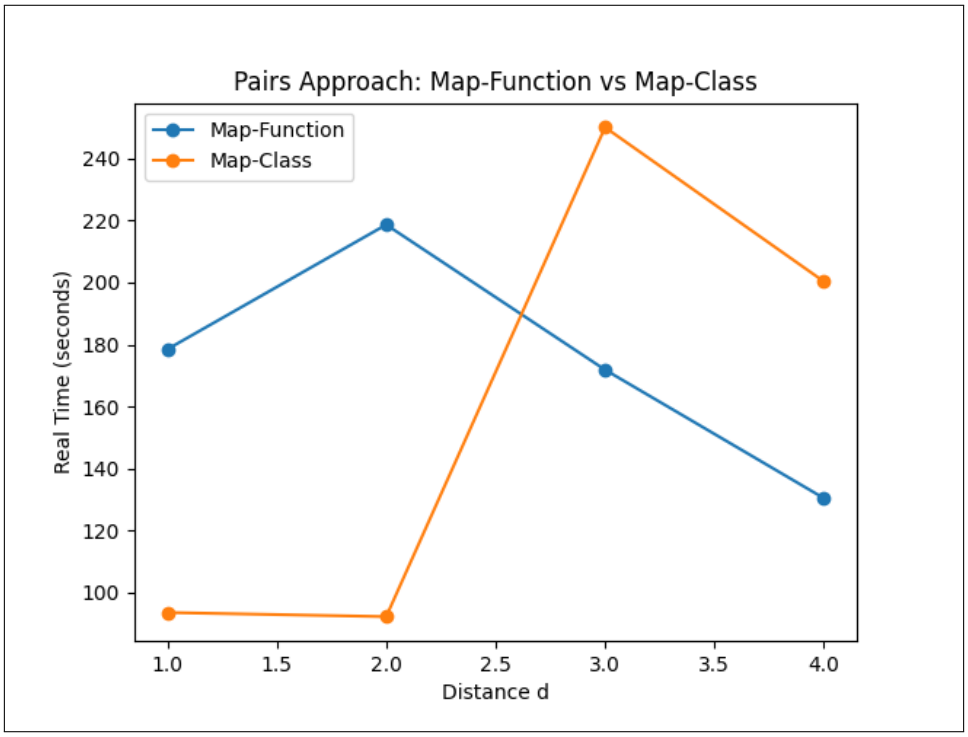


Figure 17: Pairs Approach: Map-Function vs. Map-Class

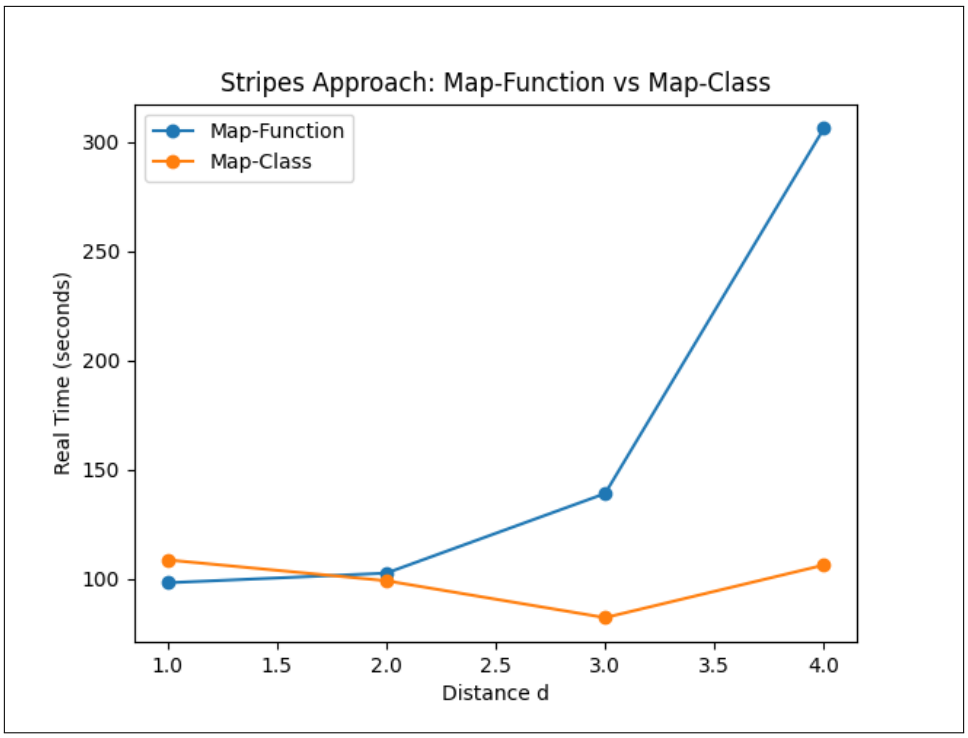


Figure 18: Stripes Approach: Map-Function vs. Map-Class

2 Problem 2: Indexing Documents via Hadoop

Objective: Compute Document Frequency (DF) for all terms in the Wikipedia corpus, then compute a TF-IDF–style score for the top-100 high-DF terms across all documents. The Porter Stemmer from `opennlp-tools-1.9.3.jar` is used for term normalisation.

2.1 Part (a): Document Frequency Computation

2.1.1 Pseudocode

Algorithm 9 Document Frequency — Map and Reduce

```
// Setup: load stopwords and initialize stemmer
function setup(context):
    stopwords ← load_from_cache()
    stemmer ← new PorterStemmer()
// Mapper
function Map(filePath, line):
    docId ← extract_filename_without_extension(filePath)
    tokens ← tokenize(line.toLowerCase())
    seenThisLine ← {}
    for each token in tokens:
        if not isValidToken(token):
            continue
        if token in stopwords:
            continue
        stemmed ← stemmer.stem(token)
        if not isValidToken(stemmed):
            continue
        if stemmed in stopwords:
            continue
        if stemmed not in seenThisLine:
            seenThisLine.add(stemmed)
            emit(stemmed, docId)
// Reducer
function Reduce(term, docIds):
    distinctDocs ← {}
    for each docId in docIds:
        distinctDocs.add(docId)
    emit(term, size(distinctDocs))
```

2.1.2 Program Logic

The Document Frequency (DF) program identifies the total number of documents (or records) in which each unique term appears across the Wikipedia corpus. Term normalization via stemming is employed to aggregate variations of the same root word.

- **Setup Phase**: The stopwords file is loaded from the *Distributed Cache* into a `HashSet` in the Mapper's `setup()` method. A `PorterStemmer` instance is also created for term normalization.
- **Mapper**: The mapper derives a document ID from the input filename by stripping the file extension. Each input line is converted to lowercase, split on non-alphabetic characters, and filtered for valid tokens (length > 2, alphabetic, not Roman numerals). Stopwords are removed before and after stemming. A per-line `HashSet` ensures each stemmed term is emitted at most once per line. The mapper emits `(stemmed erm, docId)` for each distinct term.
- **Reducer**: For each stemmed term, the reducer collects all document IDs, deduplicates them using a `HashSet`, and writes the term with its distinct document frequency count. This produces the final DF values for the corpus.
- **Post-processing**: The resulting TSV is sorted numerically by DF in descending order using shell commands to identify the top-100 terms.

2.1.3 Post-processing for Top-100 Terms

The following shell command extracts the top-100 highest document-frequency terms by sorting the MapReduce output:

```
hdfs dfs -cat /problem2/2a/df.tsv | sort -k2 -nr | head -100 >
    top100.txt
```

2.1.4 Runtime Analysis

```
real 1m56.062s  user 2m11.019s  sys 0m9.638s
```

Runtime Behavior
The execution time for document frequency computation is observed to be approximately 1 minute and 56 seconds. The user time (2 minutes and 11 seconds) is higher than the real time, indicating that multiple map and reduce tasks are executing in parallel, resulting in CPU usage exceeding the wall-clock time.

Computation vs Framework Overhead
The relatively high user time reflects the computational complexity involved in tokenization, stopword removal, stemming, and tracking unique document occurrences for each term. These operations require significant CPU processing across multiple nodes or threads.

Parallelism Effect
At the same time, the difference between real time and user time suggests that Hadoop effectively utilizes parallelism to distribute the workload.

System Overhead
The system time remains relatively low (around 9 seconds), indicating that operating system-level operations such as file handling and process management do not contribute significantly to the overall execution time.

2.1.5 Output Schema

The output TSV file has the following schema:

`TERM\tDF`

2.1.6 Top-100 High Document Frequency Terms

```
siddharth@DESKTOP-FESEIHA:~/Hadoop_Project/Assig2/Prob2$ cat top100-df.tsv
apo     9518
categori        9381
refer   9340
quot    8674
other   8415
link    8296
thi     8190
extern  8008
two     7949
includ  7885
year    7801
first   7689
http    7418
time    7349
state   7299
name    6838
www     6804
see     6761
histori 6662
peopl   6626
part    6504
unit    6500
made    6332
known   6085
place   5973
gener   5828
call    5804
well    5660
world   5640
live    5627
follow  5615
three   5601
american        5501
earli   5473
both    5454
work    5427
origin  5402
through 5381
nation  5378
present 5378
```

Figure 19: Pairs Map-Function aggregation, $d = 3$

2.2 Part (b): TF-IDF Score Computation – Stripes Approach

2.2.1 Pseudocode

Algorithm 10 TF-IDF Score — Stripes MapReduce

```
// Setup: load DF map from Distributed Cache (Mapper & Reducer)
function setup(context):
    df_map ← load_tsv_as_map(cache_file)                    ▷ Maps term → DF
// Mapper (Stripes)
function Map(filePath, line):
    docID ← extract_filename_without_extension(filePath)
    stripe ← docStripes[docID] if exists else {}
    tokens ← tokenize(line.toLowerCase())
    for each token in tokens:
        stemmed ← stem(token)
        if stemmed in df_map:
            stripe[stemmed] += 1
    docStripes[docID] ← stripe
function cleanup(context):
    for each (docID, stripe) in docStripes:
        stripeString ← serialize_stripe(stripe)             ▷ term:count|term:count|...
        emit(docID, stripeString)
// Reducer
function Reduce(docID, stripeStrings):
    termCounts ← {}
    for each stripeString in stripeStrings:
        pairs ← split(stripeString, "|")
        for each pair in pairs:
            (term, count) ← split(pair, ":")
            termCounts[term] += parseInt(count)
    for each (term, tf) in termCounts:
        df ← df_map[term]
        score ← tf * log(10000.0 / df + 1)
        emit(docID, term + "\t" + format(score, 6))
```

2.2.2 Program Logic

The Stripes approach is used to efficiently compute the TF-IDF scores for the top-100 high-DF terms across Wikipedia documents.

- **Setup Phase**: Both the Mapper and the Reducer load the `df_top100.tsv` file from the *Distributed Cache*. This populates a `dfMap` in each task, containing only the top-100 terms and their document frequencies.
- **Mapper**: The mapper derives the document ID from the input filename by stripping the file extension. For each input line, it tokenizes and lowercases text, stems each token, and checks whether the stemmed term is in `dfMap`. If so, it increments the count in an in-memory stripe for that document. No output is

emitted inside `map()`; instead, the mapper emits one stripe per document during `cleanup()`.

- **Stripe Format**: Each emitted stripe is serialized as a string of the form `term1:count1|term2:co` where `|` separates term-count pairs and `:` separates term and count.
- **Reducer**: For each document ID, the reducer receives one or more stripe strings. It merges these partial stripes into a single term-count map, computes TF-IDF scores using `SCORE = TF * log(10000 / DF + 1)`, and emits output in the schema `docId`.

2.2.3 Scoring Formula

$$\text{SCORE}(t, d) = \text{TF}(t, d) \times \log\left(\frac{10000}{\text{DF}(t)} + 1\right)$$

where $\text{TF}(t, d)$ is the count of term t in document d, and $\text{DF}(t)$ is the number of distinct documents containing t across the entire corpus.

2.2.4 Runtime Analysis

```
real 0m52.046s  user 0m59.602s  sys 0m6.379s
```

Runtime Behavior:
The execution time for TF-IDF score computation is observed to be approximately 52 seconds, which is significantly lower than that of document frequency computation. This improvement is primarily because document frequency operates on the entire vocabulary of the dataset, whereas TF-IDF is restricted to a smaller subset of top-100 terms, thereby reducing the input size and computational workload.

Efficiency of Stripes Approach:
The use of the stripes approach enables efficient aggregation of term frequencies, minimizing the number of intermediate key-value pairs and reducing shuffle overhead. The user time (59 seconds) is slightly higher than the real time (52 seconds), indicating that multiple MapReduce tasks are executed in parallel, leading to cumulative CPU usage exceeding the wall-clock time.

2.2.5 Output Schema

The output TSV file has the following schema:

`ID\tTERM\tSCORE`

The `ID` field uses dynamically generated integer IDs derived from the byte offset of the record (e.g., `DOC_12345`).

2.2.6 Representative TF-IDF Output

```
cat: Unable to write to output stream.
siddharth@DESKTOP-FESEIHA:~/Hadoop_Project/Assig2/Prob1/1a$ hdfs dfs -cat /Problem2/2b/part-r-*
100005  american        1.035975
100005  extern  0.810375
100005  three   1.024390
100005  major   1.121527
100005  www     4.520531
100005  decemb  1.204089
100005  press   3.562563
100005  group   4.779429
100005  area    1.096882
100005  apo     7.899676
100005  august  2.429295
100005  work    2.089465
100005  gener   0.999107
100005  remain  1.242151
100005  develop 1.197756
100005  both    1.041519
100005  take    1.211492
100005  includ  0.819000
100005  name    0.901143
100005  http    11.096731
100005  john    2.277968
100005  histori 3.666844
100005  quot    3.067209
100005  present 1.050621
100005  other   3.132599
100005  south   1.229167
100005  north   4.802679
100005  book    1.159151
100005  link    1.581818
100005  famili  5.675503
100005  univers 2.146793
100005  two     2.443465
100005  novemb  1.225585
100005  long    3.631496
100005  life    1.185466
100005  result  2.416379
100005  same    2.337999
100005  found   2.160831
100005  world   3.059843
100005  known   1.944121
100005  member  2.412107
100005  larg    2.346291
100005  place   2.950951
100005  make    1.089758
100005  org     2.455941
100005  categori        1.451214
100005  earli   9.353427
100005  june    1.199859
100005  thi     6.383665
100005  form    3.248204
100005  refer   0.727869
100005  time    1.717941
100005  first   0.833152
100089  year    0.825003
100089  nation  6.303728
100089  american        1.035975
100089  extern  0.810375
100089  april   1.155859
100089  four    1.212320
100089  decemb  1.204089
100089  state   2.588734
100089  press   4.750084
100089  group   1.194857
100089  area    7.678174
100089  apo     12.926743
100089  work    3.134198
100089  gener   1.998213
100089  develop 9.582044
100089  list    5.694920
100089  establish       2.401339
```

Figure 20: TF-IDF Output

Appendix

A. System Configuration

Parameter	Value
Hadoop Ver.	3.4.3
Java Ver.	Java 8
Dataset	Wikipedia-EN-20120601_ARTICLES.tar.gz
Testing Dataset	Wikipedia-50-Articles.tar

Table 8: Experimental System Configuration

B. Run Instructions

- `problem_1.txt` – Instructions for Problem 1 (parts a, b, c, e)
- `problem_2.txt` – Instructions for Problem 2 (parts a, b)

The solution is submitted in a ZIP file, and the same contents are also available in the GitHub repository linked with this assignment.

D. Shared Tokenization and Normalization

The tokenization logic is kept consistent across all MapReduce implementations in this assignment. Each line is converted to lowercase and split using the regular expression `[^a-z]+`, which removes every non-alphabetic separator. After splitting, tokens are filtered to remove:

- tokens of length two or less,
- non-alphabetic tokens,
- Roman numerals (e.g., `i`, `ii`, `iii`), and
- stopwords loaded from `stopwords.txt` via the distributed cache.

The remaining tokens are then stemmed using the OpenNLP Porter stemmer before further processing. This shared normalization ensures consistent term matching across both Problem 1 and Problem 2 implementations.